

TIPS: pdf converters (including OCR)

NLP Suite - File content checker & converter & cleaner

The NLP Suite offers three different ways of converting a pdf file. They are NOT interchangeable: which one you need depends on how your pdf was made and on what you want to do with the result. This TIPS file explains how to choose, and how to install the extra software the OCR option requires.

Which converter do I need?

Start with one question: IS YOUR PDF SCANNED?

Open the pdf in any pdf reader and try to select a few words with the mouse.

- If you CAN select the words, the pdf contains real text. Use pdfminer (pdf --> txt) or the pdf --> docx converter.
- If you CANNOT select anything, the page is just a picture and the pdf holds no text at all. Only OCR can read it.

Converter	Use it when	Output	Extra software
pdf --> txt (via pdfminer)	Your pdf has real text and you want to ANALYZE it with the NLP Suite tools	txt	none
pdf --> txt (SCANNED pdf, via pytesseract OCR)	Your pdf is a scan: you cannot select any text in it	txt	yes - see below
pdf --> docx (text & images)	You want to KEEP the whole document, pictures included, not just analyze its words	docx	pypdf module

TEXT is the only file type the NLP Suite tools can process. If your goal is to run any NLP analysis, you want a txt output.

Document converter (pdf --> txt) (via pdfminer)

The standard converter. It reads the text that is already stored inside the pdf, so it is exact and fast, and it also handles column-based pdf files (e.g., journal articles).

What you should know:

- Any IMAGE embedded in the pdf is LOST. Only the text is written out. Use the pdf -> docx converter if you need the pictures.
- Full-page newspaper articles, with several columns, headings and pictures mixed together, are unlikely to convert cleanly. Simpler layouts and books convert very well.

- The pdf breaks its lines wherever the page ran out of room, and those breaks are kept in the txt output. See "After converting" below.
- ALWAYS check the converted file. Before parsing it, consider removing copyright notices and bibliographies: they are not made of complete sentences and will distort your results.

Document converter (pdf --> txt) (SCANNED pdf, via pytesseract OCR)

OCR (Optical Character Recognition) is for pdf files that contain NO text, only a picture of each page - typically anything produced by a scanner or a photocopier, and most historical documents and newspaper archives.

The NLP Suite renders every page as a 300 dpi image and then reads the letters out of that image with Tesseract, the open-source OCR engine developed at HP and now maintained by Google.

What OCR requires

Unlike every other converter, OCR needs software that the NLP Suite installers CANNOT bundle, because it consists of separate programs and not of Python code. FOUR pieces are needed:

- the Python module pytesseract - it talks to the Tesseract program;
- the Python module pdf2image - it turns each pdf page into an image;
- the Tesseract program - it does the actual character recognition;
- the poppler program - pdf2image uses it to render the pages.

The NLP Suite checks all four when you select the OCR option and tells you exactly which ones are missing, with the command to install them. Nothing will run until all four are in place.

Installing on WINDOWS

1. In the command prompt, activate the NLP environment and install the two Python modules:

```
conda activate NLP
pip install pytesseract pdf2image
```

2. Download the Tesseract installer from

<https://github.com/UB-Mannheim/tesseract/wiki>

and install it in the default location, C:\Program Files\Tesseract-OCR. During the installation, under "Additional language data", tick every language you work with (e.g., Italian) - see "Languages" below.

3. Download poppler from

<https://github.com/oschwartz10612/poppler-windows/releases>

and unzip it to C:\Program Files\poppler.

4. Close the NLP Suite and start it again.

Installing on MAC

1. In terminal, activate the NLP environment and install the two Python modules:

```
conda activate NLP
```

```
pip install pytesseract pdf2image
```

2. Still in terminal, install the two programs with Homebrew:

```
brew install tesseract poppler
```

If you also work in a language other than English, install every language pack with:

```
brew install tesseract-lang
```

3. Close the NLP Suite and start it again.

You do NOT need to add anything to your PATH: the NLP Suite looks for Tesseract and poppler in the standard installation folders as well.

Languages

OCR must know which language it is reading: the same shapes are read differently in different languages, and accented characters will be missed if the wrong language is used. The NLP Suite passes the corpus language you selected in the Setup "NLP package and language" GUI straight to Tesseract.

If the language pack for your corpus language is not installed on your machine, the NLP Suite warns you, names the pack that is missing and carries on with what IS installed, rather than aborting the run. If you see that warning, install the missing pack (see the installation steps above) and run the conversion again.

SEVERAL languages at once

Tesseract can read more than one language in a single pass, which is what you want for a corpus that mixes languages, or for documents with quotations in another language. The "Corpus language" setting of the NLP Suite is a LANGUAGE(S) field: select every language your corpus uses and the NLP Suite passes them all to Tesseract, joined as, for instance, eng+ita. The first language is the primary one.

Do not, however, ask for languages you do not actually need. Every additional language slows the recognition down and, for languages that do not share the same alphabet, can make the result WORSE, not better.

What to expect

- OCR is SLOW: expect roughly a few seconds per page. A few hundred pages will take a good while, and the GUI will look frozen while it works. Watch the progress in the command prompt/terminal window.
- OCR is NEVER perfect. The quality of the result depends almost entirely on the quality of the scan. ALWAYS read through the converted file before analyzing it.
- If a pdf is not scanned, do NOT use OCR on it: the pdfminer option is both exact and far faster.

If the output txt file is EMPTY

The NLP Suite tells you which files produced no text at all. The usual causes are:

- the scan is too poor, too dark or too low-resolution to be read;

- the pages are upside down or sideways - rotate them in a pdf reader and save, then try again;
- the language pack does not match the language of the document.

Document converter (pdf --> docx) (text & images)

This converter keeps BOTH the text and the images embedded in your pdf files in a single Word document. Use it when you want to preserve the whole document rather than analyze its words - a txt file cannot hold a picture.

It also keeps the bold, the italics, the type size and the colour used in the pdf, and it joins the pdf line breaks back into flowing sentences.

The docx output is a REFLOW of the pdf and NOT a photographic copy.

You get the original text and images, in reading order, in a single column. You do NOT get the original fonts, columns and page geometry. Tables come out as lines of text and not as Word tables. A pdf that is a scan contains no text to extract, so only its images are saved.

This option needs the Python module pypdf, which decodes the images. If it is missing, the NLP Suite tells you the command to install it:

```
conda activate NLP
pip install pypdf
```

An image that cannot be decoded is never dropped silently: the docx will carry a visible [IMAGE could not be extracted] line exactly where the picture belonged.

If you need the TABLES in a pdf

None of the three converters treats a table as a table: in a txt file, and in the docx output, the cells of a table come out as ordinary lines of text, and the rows and columns are lost. If your research material is tabular - statistical appendices, historical series, catalogues - you will want the tables as csv files, one row per row and one column per column, which you can then feed straight back into the many csv tools of the NLP Suite.

The NLP Suite does not do this for you. It is a specialised need, and the library that does the job well would add a compiled component to every installation for the sake of a minority of users. It is, however, a five-minute job to do it yourself, and the library is free and open source (MIT licence).

1. Install the library. In command prompt/terminal:

```
conda activate NLP
pip install pdfplumber
```

2. Save the following few lines in a file called extract_tables.py (any plain text editor will do; do NOT use Word):

```
import csv, os, sys
import pdfplumber
```

```

pdf_file = sys.argv[1]          # the pdf to read
output_dir = sys.argv[2]        # where to write the csv files

with pdfplumber.open(pdf_file) as pdf:
    for page_number, page in enumerate(pdf.pages, start=1):
        for table_number, table in
enumerate(page.extract_tables(), start=1):
            name = 'table_page%d_%d.csv' % (page_number,
table_number)
            with open(os.path.join(output_dir, name), 'w',
newline='',
                        encoding='utf-8') as f:
                csv.writer(f).writerows(table)
            print('wrote', name)

```

3. Run it, giving it the pdf and the directory where the csv files should go:

```
python extract_tables.py "C:\my corpus\report.pdf" "C:\my output"
```

You get one csv file per table, named after the page it came from (table_page1_1.csv, table_page1_2.csv, ...). From there, every csv tool of the NLP Suite applies: the csv to txt converter of this same GUI, the statistical tools, the data manipulation tools.

Check the csv files. Table extraction works well on tables that are ruled, or whose columns are cleanly aligned. Tables held together only by white space, and tables with merged or nested cells, come out imperfectly and may need hand correction. As always, the cleaner the original, the better the result.

Input and output

- In INPUT you can select either a single pdf file or a whole directory. When you select a directory, the script asks whether you also want to convert the files in its subdirectories.
- Files whose name does not end in .pdf are ignored; .PDF in capitals is accepted.
- In OUTPUT the converted files are written to the OUTPUT directory you selected, in the same subdirectory structure as the input files. The pdf --> docx and OCR converters place their results in a subdirectory of their own (pdf_2_docx, pdf_OCR_2_txt).
- One unreadable pdf no longer stops a whole batch: the run continues and every file that failed is listed at the end.

After converting: cleaning the txt output

A pdf breaks its lines wherever the page ran out of room. In a txt file those breaks become real line breaks, sometimes in the middle of a word. Before parsing, use the "Clean Files" options of this same GUI:

- Remove end-of-line typesetting hyphenation and join split parts - joins a word split across two lines. Note that it will ask you first: an end-of-line hyphen can also be a legitimate compound (pretty-smart), which no algorithm can tell apart.
- Remove all end-of-line hard carriage returns - puts the sentences back together.
- Change to ASCII non-ASCII apostrophes & quotes and % to percent - IMPORTANT: a % sign breaks the Stanford CoreNLP parser.
- Remove blank lines from text file(s).

Further readings: More TIPS files

TIPS_NLP_File checker & converter & cleaner.pdf

TIPS_NLP_File handling in NLP Suite.pdf

TIPS_NLP_Anaconda NLP environment pip.pdf